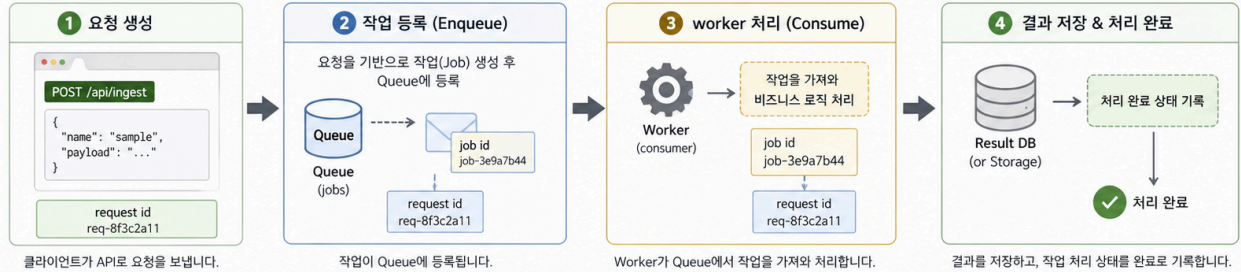


4교시: Branch 전략 - dev, stage, prod

Week 3 Day 3 Lesson 4 – worker/queue 실습

요청 → 작업 등록 → worker 처리 → 결과 저장 → 로그 확인까지의 전체 흐름을 실습합니다.



처리 로그 (예시)

```
2025-06-09T11:20:01.123Z INFO [api] request received request_id=req-8f3c2a11 path=/api/ingest
2025-06-09T11:20:01.125Z INFO [api] job enqueued request_id=req-8f3c2a11 job_id=job-3e9a7b44
2025-06-09T11:20:01.126Z INFO [api] response 202 Accepted request_id=req-8f3c2a11
2025-06-09T11:20:02.345Z INFO [worker] job picked up job_id=job-3e9a7b44 request_id=req-8f3c2a11
2025-06-09T11:20:02.346Z INFO [worker] processing... job_id=job-3e9a7b44 request_id=req-8f3c2a11
2025-06-09T11:20:04.789Z INFO [worker] processing done job_id=job-3e9a7b44 request_id=req-8f3c2a11
2025-06-09T11:20:04.790Z INFO [worker] result saved job_id=job-3e9a7b44 request_id=req-8f3c2a11
2025-06-09T11:20:04.791Z INFO [worker] ack (completed) job_id=job-3e9a7b44 request_id=req-8f3c2a11
```

- request_id: 클라이언트 요청 단위의 고유 ID (추적/상관관계)
- job_id: Queue에 등록된 작업 단위의 고유 ID (재처리/모니터링)

명령어 & 확인 (Evidence)

1) worker 로그 확인

```
docker compose logs -f worker
```

- 확인 포인트
- job picked up
 - processing done
 - result saved
 - request_id, job_id 일치

2) Queue 길이 확인

```
docker compose exec broker \
redis-cli LLEN jobs
```

예시 결과
8
0이면 현재 대기 중인 작업이 없음 (모두 소비되었음)

3) 처리된 작업 수 확인

```
docker compose exec db \
psql -U app -d app -t -c \
"SELECT COUNT(*) FROM results;"
```

예시 결과
15
지금까지 처리 완료되어 저장된 결과(잡)의 총 개수

전체 흐름 요약



수업 목표

- dev, stage, prod branch 전략의 장단점을 설명한다.
- branch와 environment를 분리해서 생각한다.
- promotion 방식과 drift 위험을 이해한다.

흔한 전략 1: dev/stage/prod branch

```
feature/*
-> dev
-> stage
-> prod
```

장점:

장점	설명
환경별 승인 명확	stage/prod 승인을 branch로 표현
운영팀 이해 쉬움	prod branch가 운영 기준
hotfix 경로 분리 가능	prod에서 긴급 수정 관리

단점:

단점	설명
branch drift	dev/stage/prod가 서로 달라짐
cherry-pick 증가	일부 변경만 올리려다 이력 복잡
conflict 증가	오래 유지되는 branch는 충돌 가능성 큼

전략 2: main + GitHub Environment

```
feature/*
-> main
-> deploy dev/stage/prod by environment gate
```

장점:

장점	설명
코드 이력 단순	main 중심
환경 정책 분리	GitHub Environment approval 사용
CI/CD와 잘 맞음	같은 commit을 환경별로 promotion

단점:

단점	설명
초기 설계 필요	workflow와 environment 설정 필요
조직 이해 필요	branch가 환경이라는 사고에서 전환 필요

선택 기준

상황	추천
초급 팀, 환경 승인 명확히 보여야 함	dev/stage/prod branch
CI/CD 성숙, 배포 자동화 중심	main + environment
릴리스 주기 길고 hotfix 많음	Git Flow 일부
빠른 웹 서비스	GitHub Flow

실습 질문

W3D2의 order-worker 변경을 운영에 반영한다고 가정한다.

질문	답
dev에서 먼저 확인할 것은 무엇인가	
stage에서 확인할 것은 무엇인가	
prod에 바로 들어가면 안 되는 이유는 무엇인가	
Docker image tag는 어느 단계에서 고정할 것인가	

핵심 포인트

branch는 환경을 표현할 수도 있지만, 항상 좋은 답은 아니다. 중요한 것은 같은 commit과 같은 image가 어떤 gate를 거쳐 dev/stage/prod로 올라가는지 추적하는 것이다.

Evidence Note

```
# W3D3S4 Branch Strategy
- selected model:
- dev gate:
- stage gate:
- prod gate:
- drift risk:
- image tag policy:
```